

Package ‘mcstprules’

July 10, 2026

Type Package

Title Cost Allocation Rules in Minimum Cost Spanning Tree Problems

Version 0.1.0

Description Computes, analyzes, and visualizes cost allocation rules for Minimum Cost Spanning Tree Problems (MCSTP). The package implements both algorithmic methods and rules based on cooperative game theory, alongside a comprehensive suite of analytical tools to evaluate core stability, geometric properties, and cost sensitivity.

License GPL (>= 3)

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Imports gtools,
igraph,
readxl,
CoopGame,
fields,
geometry,
lpSolve,
plotly,
rcdd

URL <https://github.com/luciasouto13/mcstprules>, <https://luciasouto13.github.io/mcstprules/>

BugReports <https://github.com/luciasouto13/mcstprules/issues>

Suggests knitr,
rmarkdown

VignetteBuilder knitr

Contents

mcstBird	2
mcstBoruvka	5
mcstCompare	7
mcstCone	8
mcstCoreCheck	10

mcstCorePlot	12
mcstCorePoints	14
mcstDuttaKar	15
mcstFolk	18
mcstGameCC	20
mcstGameIrred	24
mcstGameOpt	27
mcstGamePrivate	30
mcstGamePublic	33
mcstKar	36
mcstOWShapley	38
mcstPWShapley	40
mcstRules	42
mcstSensitivity	44
mcstStabilityRange	46

Index	48
--------------	-----------

mcstBird	<i>Bird's Rule for MCSTP</i>
----------	------------------------------

Description

This function computes Bird's cost allocation (Bird, 1976) for a minimum cost spanning tree problem (N_0, C) . The Bird rule, denoted as $B(N_0, C)$, assigns to each agent the cost of the arc that connects them to the source during the construction of a minimum cost spanning tree using Prim's algorithm.

Usage

```
mcstBird(
  C,
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node). For $n \leq 3$ with ties and method = "exact", it also displays all possible trees.
which.plot	character string indicating which plots to display (only for $n \leq 3$ with ties and method = "exact"). If "details", displays the detailed breakdown of all trees and allocations according to the agents' entry orders; if "main", the average allocation is plotted. Default is c("main", "details").

titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm, method (exact or Monte Carlo), and the total network cost.
method	character string specifying the calculation method when ties occur. If "exact" (default), computes all $n!$ permutations; if "montecarlo", estimates the average allocation through random sampling using n_{sim} permutations (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

For a given tree g^n constructed following Prim's algorithm, the allocation for each agent $i \in N$ is given by:

$$B_i(N_0, C) = c_{i^0 i},$$

where i^0 is the node to which agent i is first connected in the process of building the network.

When the minimum cost spanning tree is not unique, the rule is extended by averaging the allocations over all possible permutations $\pi \in \Pi_N$ (method = "exact"):

$$B(N_0, C) = \frac{1}{n!} \sum_{\pi \in \Pi_N} B^\pi(N_0, C),$$

where $B^\pi(N_0, C)$ is the allocation obtained by applying Prim's algorithm to (N_0, C) and solving indifferences by selecting the first agent given by the permutation π , as proposed by Dutta and Kar (2004).

Alternatively, method = "montecarlo" estimates the average allocation through random sampling using n_{sim} permutations. This is highly useful when the number of agents is large, as it significantly reduces computation time while providing a very accurate approximation:

$$B(N_0, C) \approx \frac{1}{n_{\text{sim}}} \sum_{k=1}^{n_{\text{sim}}} B^{\pi_k}(N_0, C),$$

where π_k represents each of the randomly sampled permutations. For networks with few agents, the exact method is preferred to avoid potential overestimations.

Value

A list containing:

- bird: the Bird's allocation vector $B(N_0, C)$ for the natural ordering.
- e_bird: the extended Bird's allocation (average over permutations if ties exist; equal to bird otherwise).
- arcs: if the MCST is unique, a data frame of edges (i, j) in the tree and the stage at which they were added; if ties exist, a list of such data frames, one per distinct optimal tree.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent, based on e_bird.
- ranking: a ranking of agents by cost in e_bird (from highest to lowest; ties marked with *).

- `perms`: if ties exist, a list with `$table` (allocations for at most the first 6 permutations computed, regardless of n), `$summary` (distinct allocations with their multiplicity, sorted in decreasing order) and `$nperms` (total number of permutations computed); NULL if the MCST is unique.
- `is_unique`: logical; if TRUE, the MCST is unique.
- `method`: the method applied for computing the extended rule ("exact" or "montecarlo").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where `C[i, j]` represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335–350.

Dutta B, Kar A (2004) Cost monotonicity, consistency and minimum cost spanning tree games. *Games Econom Behav* 48(2):223–248.

See Also

[mcstDuttaKar](#) for other rule based on Prim's algorithm.

[mcstGameIrred](#) for a cooperative game whose Shapley value coincides with Bird's solution when applied to the irreducible matrix $\bar{C}$.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstBird(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstBird(C_inf)

# Matrix input with ties
```

```

C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstBird(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)

```

mcstBoruvka

Boruvka's Rule for MCSTP

Description

This function computes Boruvka's cost allocation (Bergantiños and Vidal- Puga, 2011) for a minimum cost spanning tree problem (N_0, C) . Boruvka's rule, denoted as $\beta^\pi(N_0, C)$, is defined through Boruvka's algorithm.

Usage

```
mcstBoruvka(C, draw = FALSE, titles = TRUE)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

The Boruvka allocation divides the cost of the optimal tree through a step-by-step process over γ stages. At each stage s , agents pay the maximum possible proportion p^s of the cheapest arc associated with their connected component.

Let A^s be the set of non-completely paid arcs, ϱ_{ij}^s the proportion of the cost of arc (i, j) already paid, and N_{ij}^s the set of agents paying for arc (i, j) in stage s . The proportion p^s is given by:

$$p^s = \min \left\{ \frac{1 - \varrho_{ij}^{s-1}}{|N_{ij}^s|} : (i, j) \in A^{s-1}, N_{ij}^s \neq \emptyset \right\}.$$

Let a_i^s be the arc partially paid by agent i in stage s . The cost paid by agent i in this stage is:

$$f_i^s = p^s c_{a_i^s}.$$

The process finishes in γ stages when the tree is completely paid, i.e., $\sum_{s=1}^{\gamma} p^s = 1$. The Boruvka allocation for each agent $i \in N$ is given by:

$$\beta_i^\pi(N_0, C) = \sum_{s=1}^{\gamma} f_i^s.$$

Value

A list containing:

- **boruvka**: the Boruvka's allocation vector $\beta^\pi(N_0, C)$.
- **arcs**: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- **total**: the total cost of the MCST, $m(N_0, C)$.
- **percentage**: the share of the total cost allocated to each agent.
- **ranking**: a ranking of agents by cost (from highest to lowest; ties marked with *).
- **is_unique**: logical; if TRUE, the MCST is unique.

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

Boruvka's allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Boruvka's algorithm yields the same cost distribution.

Furthermore, as demonstrated by Bergantiños and Vidal-Puga (2011), for any order π used in the algorithm, Boruvka's allocation is equivalent to the [mcstFolk](#).

References

Bergantiños G, Vidal-Puga J (2011) The folk solution and Boruvka's algorithm in minimum cost spanning tree problems. *Discrete Appl Math* 159(12):1279–1283

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[mcstFolk](#) for an equivalent rule based on Kruskal's algorithm.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstBoruvka(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstBoruvka(C_inf)

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstBoruvka(C_mat, draw = TRUE, titles = FALSE)
```

mcstCompare

Compare Allocation Rules for MCST

Description

This function computes and compares the cost allocations for a minimum cost spanning tree (MCST) problem using four standard rules: Bird, Dutta-Kar, folk, and Kar.

Usage

```
mcstCompare(C)

## S3 method for class 'mcstp_compare'
plot(x, titles = TRUE, ...)
```

Arguments

C	a cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" in mcstBird).
x	the output from mcstCompare.
titles	logical; if TRUE (default), adds informative main titles and axis labels to the plot.
...	additional graphical parameters passed to barplot .

Value

A data frame containing the allocations for each agent under the Bird, Dutta-Kar, folk, and Kar rules.

See Also

[mcstBird](#), [mcstDuttaKar](#), [mcstFolk](#), [mcstKar](#) for the rule definitions.
[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Matrix input
C_mat <- matrix(c(0, 10, 15, 20,
                  10, 0, 25, 12,
                  15, 25, 0, 8,
                  20, 12, 8, 0), byrow = TRUE, ncol = 4)

# Compare all rules
comparison <- mcstCompare(C_mat)
comparison

# Plot the comparison
plot(comparison)
```

mcstCone

Cone-wise Decomposition Rules for MCSTP

Description

This function computes cost allocations for minimum cost spanning tree problems (N_0, C) using the cone-wise decomposition introduced by Norde et al. (2004). This technique allows extending rules defined for elementary MCSTPs to any MCSTP.

Usage

```
mcstCone(
  C,
  rule = c("folk", "owshapley", "bogomolnaia"),
  weights = NULL,
  lambda = 1
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
rule	a character string indicating the rule R to be applied to the elementary MCSTPs. One of "folk" (default), "owshapley", or "bogomolnaia".
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents; only if rule = "owshapley".
lambda	a non-negative numeric parameter (1 by default); only if rule = "bogomolnaia".

Details

Any MCSTP can be written as a nonnegative combination of elementary MCST $C = \sum_{q=1}^{m(C)} x^q C^q$, where the costs of the arcs in C^q are 0 or 1. A rule R is extended as:

$$R(N_0, C) = \sum_{q=1}^{m(C)} x^q R(N_0, C^q).$$

Let g^q be the network at stage $q \in \{1, \dots, m(C)\}$, and $S_i^q := S(P(g^q), i)$ the connected component containing agent i in the graph induced by zero-cost arcs. For each elementary problem (N_0, C^q) , the agents in a component S not containing the source (0) divide the cost of connecting to the source according to the chosen rule:

- Folk rule ("folk"):

$$F(N_0, C^q) = \begin{cases} \frac{1}{|S_i^q|} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise.} \end{cases}$$

- Optimistic weighted Shapley rule ("owshapley"):

$$f^{\text{owshapley}}(N_0, C^q) = \begin{cases} \frac{w_i}{\sum_{j \in S_i^q} w_j} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise.} \end{cases}$$

- Bogomolnaia and Moulin family ("bogomolnaia"):

For $\lambda \in [0, +\infty)$,

$$R^\lambda(N_0, C^q) = \begin{cases} \frac{\lambda^{\delta_i}}{\sum_{j \in S_i^q} \lambda^{\delta_j}} & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise,} \end{cases}$$

where δ_i denotes the number of non-null arcs in C^q containing agent i .

For $\lambda = +\infty$,

$$R^\lambda(N_0, C^q) = \begin{cases} \arg \max_{j \in S_i^q} \delta_j & \text{if } 0 \notin S_i^q \\ 0 & \text{otherwise,} \end{cases}$$

i.e., agents attaining the maximum value of δ_j share the cost equally.

Value

A list containing:

- allocation: the cost allocation vector $R(N_0, C)$ for the chosen rule R .
- rule: the rule R applied for the allocation.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- decomposition: a data frame containing the details of the cone-wise decomposition, including the cost threshold, the weight of each layer and the elementary allocation for each agent.
- weights: the weight vector used for the allocation; only if rule = "owshapley".
- lambda: the lambda value used for the allocation; only if rule = "bogomolnaia".

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

This function focuses on the analytical and numerical decomposition of the cost allocation. For graphical representations, use the direct rule functions (e.g., [mcstFolk](#)).

The allocation for rule = "folk" is also obtained when all weights are equal in rule = "owshapley", or when $\lambda = 1$ in rule = "bogomolnaia".

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Norde H, Moretti S, Tijs S (2004) Minimum cost spanning tree games and population monotonic allocation schemes. *Eur J Oper Res* 154(1):84–97

See Also

[mcstFolk](#), [mcstOWShapley](#) for direct implementations of these rules.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
C <- c(12, 15, 20, 4, 6, 8)

# Folk rule
mcstCone(C, rule = "folk")

# Optimistic weighted Shapley rule
mcstCone(C, rule = "owshapley", weights = c(2, 1, 1))

# Bogomolnaia and Moulin family
mcstCone(C, rule = "bogomolnaia", lambda = 2)
mcstCone(C, rule = "bogomolnaia", lambda = Inf)
```

mcstCoreCheck

Core Stability Check for Cost Allocations

Description

This function evaluates whether a given cost allocation belongs to the *core* of a cooperative game. It checks for both efficiency and coalitional rationality, identifying any blocking coalitions if the allocation is unstable.

Usage

```
mcstCoreCheck(game = NULL, allocation = NULL, v = NULL, tol = 1e-07)
```

```
## S3 method for class 'mcstp_core'
plot(x, col = "firebrick", titles = TRUE, ...)
```

Arguments

game	list; an object containing a cooperative game computed with method = "exact" (e.g., from <code>mcstGamePrivate</code>). Default is NULL.
allocation	numeric vector; a proposed cost allocation for each agent. Required if game is NULL.
v	numeric vector; the characteristic function values $v(S)$ for all $2^n - 1$ coalitions, ordered by cardinality. Required if game is NULL.
tol	numeric; tolerance for floating-point comparisons to avoid precision issues. Default is $1e-7$.
x	the output from <code>mcstCoreCheck</code> .
col	character string specifying the color for the excess bars. Default is "firebrick".
titles	logical; if TRUE (default), adds a main title and a y-axis label to the plot.
...	additional graphical parameters passed to <code>barplot</code> .

Details

An allocation $x \in \mathbb{R}^N$ belongs to the *core* of a cooperative game (N, v) if it satisfies two fundamental conditions:

1. Efficiency: the total cost is completely allocated among the agents, i.e.,

$$\sum_{i \in N} x_i = v(N).$$

2. Coalitional rationality: no coalition pays more than its stand-alone cost, i.e.,

$$\sum_{i \in S} x_i \leq v(S) \quad \forall S \subset N.$$

If the coalitional rationality condition is violated for any coalition S , S is considered a *blocking coalition*, as its members would have an incentive to leave the grand coalition and form their own sub-network. The excess $x(S) - v(S)$ quantifies the magnitude of this violation.

The function can be executed in two ways: by passing a complete game object (which must have been computed with method = "exact"), or by manually providing the proposed allocation vector and the characteristic function v .

The plot method visualizes the excess of the blocking coalitions (if any) using a bar chart.

Value

A list containing:

- `in_core`: logical; if TRUE, the allocation is stable (belongs to the *core*).
- `is_efficient`: logical; if TRUE, the sum of the allocation equals the total cost.

- `is_rational`: logical; if TRUE, no blocking coalitions exist.
- `blocking_coals`: a data frame detailing the blocking coalitions S , their allocated cost $x(S)$, their value $v(S)$, and the excess. Empty if the allocation is rational.
- `n`: the number of agents.
- `sum_x`: the sum of the allocated costs.
- `total`: the total cost of the game $v(N)$.

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[mcstGamePrivate](#), [mcstGameIrred](#) for computing cooperative games that can be passed to this function.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Example 1: Using manual vectors
vvals <- c(10, 12, 15, 20, 22, 25, 30) # 3 agents: v(1), v(2), v(3), v(12), ...
# Stable check
alloc1 <- c(8, 10, 12)
check1 <- mcstCoreCheck(allocation = alloc1, v = vvals); check1
# Unstable check
alloc2 <- c(15, 10, 5)
check2 <- mcstCoreCheck(allocation = alloc2, v = vvals); check2
plot(check2)

# Example 2: Using a game object
C <- matrix(c(0, 12, 15, 12, 0, 4, 15, 4, 0), byrow = TRUE, ncol = 3)
# Check if the Shapley value of the private game is in the core
mcstCoreCheck(game = mcstGamePrivate(C))
```

mcstCorePlot

Plot the Core Region and Cost Allocations

Description

This function generates a geometric visualization of the *core* region and the imputation set for cooperative games with 2, 3, or 4 players. It also allows plotting multiple cost allocation rules simultaneously to visually check their stability.

Usage

```
mcstCorePlot(game = NULL, v = NULL, allocations = list(), titles = TRUE)
```

Arguments

game	list; an object containing a cooperative game (e.g., from <code>mcstGamePrivate</code>). If provided and <code>allocations</code> is empty, its allocation and solution concept are used by default. Default is <code>NULL</code> .
v	numeric vector; the characteristic function values $v(S)$ for all $2^n - 1$ coalitions, ordered by cardinality. Required if game is <code>NULL</code> .
allocations	list; a named list of numeric vectors representing different cost allocations to be plotted as points. Default is <code>list()</code> .
titles	logical; if <code>TRUE</code> (default), adds a main title and subtitle/legend information to the plot.

Details

The geometric representation of the *core* and the imputation set depends on the number of players n :

1. $n = 2$ players: the imputation set is represented as a 1D line segment where $x_1 + x_2 = v(N)$. The *core* is highlighted as a sub-segment within it, delimited by individual rationality.
2. $n = 3$ players: allocations are projected into 2D barycentric coordinates (simplex). The imputation set forms a large triangle, and the *core* is plotted inside as a bounded convex polygon.
3. $n = 4$ players: the function switches to an interactive 3D plot using the `plotly` package. The imputation set is drawn as a transparent 3D tetrahedron (mesh), and the *core* is represented as a solid 3D polytope inside it.

If an allocation lies inside the green *core* region, it means the allocation is coalitionally stable (no group of players has incentives to defect).

The function automatically extracts the default allocation if a game object is passed. Alternatively, a custom named list of allocations can be provided to compare different rules (e.g., Bird, Shapley value, folk) on the same graph.

Value

For $n = 2$ or $n = 3$, the function draws a base R plot. For $n = 4$, it returns an interactive `plotly` object containing the 3D visualization.

See Also

[mcstCoreCheck](#) for checking stability numbers and finding blocking coalitions.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Example 1
vvals1 <- c(10, 12, 15, 20, 22, 25, 30) # 3 players: v(1), v(2), v(3), v(12), ...
allocs1 <- list(RuleA = c(8, 10, 12), RuleB = c(9, 10, 11), RuleC = c(7, 11, 12))
mcstCorePlot(v = vvals1, allocations = allocs1)

# Example 2 (interactive 3D plot)
vvals2 <- c(10, 10, 10, 10, 18, 18, 18, 18, 18,
            18, 25, 25, 25, 25, 32) # 4 players: v(1), v(2), v(3), v(4), v(12), ...
allocs2 <- list(RuleA = c(8, 8, 8, 8), RuleB = c(10, 6, 11, 5))
```

```
mcstCorePlot(v = vvals2, allocations = allocs2)
```

mcstCorePoints	Check Core Emptiness
----------------	----------------------

Description

This function determines whether the *core* of a cooperative game is empty by solving a Linear Programming (LP) problem. If the *core* is non-empty, it returns a feasible stable allocation and computes all the geometric vertices of the *core* polytope.

Usage

```
mcstCorePoints(game = NULL, v = NULL)

## S3 method for class 'mcstp_core_point'
plot(x, titles = TRUE, ...)
```

Arguments

game	list; an object containing a cooperative game computed with method = "exact" (e.g., from mcstGamePrivate). Default is NULL.
v	numeric vector; the characteristic function values $v(S)$ for all $2^n - 1$ coalitions, ordered by cardinality. Required if game is NULL.
x	the output from mcstCorePoints.
titles	logical; if TRUE (default), adds a main title and subtitle/legend information to the plot.
...	additional graphical parameters passed to mcstCorePlot .

Details

The problem of finding a cost allocation $x \in \mathbb{R}^N$ in the *core* is formulated as a Linear Programming system. The allocation must satisfy two fundamental conditions:

1. Efficiency: the total cost is completely allocated among the agents, i.e.,

$$\sum_{i \in N} x_i = v(N).$$

2. Coalitional rationality: no coalition pays more than its stand-alone cost, i.e.,

$$\sum_{i \in S} x_i \leq v(S) \quad \forall S \subset N.$$

The function uses the lpSolve package to evaluate this system of equations and inequalities. If a solution exists, the *core* is non-empty and a feasible allocation is returned.

Furthermore, for games with up to 6 players ($n \leq 6$), the function automatically utilizes the rcdd package to perform vertex enumeration. It converts the system of linear inequalities into the exact geometric vertices of the *core*. This is particularly useful for analyzing the bounds of the *core* region.

The plot method provides a visualization of the *core* and the computed feasible point by internally calling [mcstCorePlot](#). It is supported for games with 2, 3, or 4 players.

Value

A list containing:

- `is_empty`: logical; if TRUE, the *core* is empty (no stable allocation satisfies all conditions).
- `core_point`: numeric vector; a feasible allocation inside the *core*. NULL if the *core* is empty.
- `vertices`: a matrix where each row represents a geometric vertex of the *core* polytope. NULL if the *core* is empty or $n > 6$.
- `n`: the number of agents.
- `total`: the total cost of the game $v(N)$.
- `values`: the characteristic function values $v(S)$.

See Also

[mcstCoreCheck](#) for checking the stability of a specific allocation.

[mcstCorePlot](#) for general *core* region visualizations.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Example 1: Non-empty core
vvals1 <- c(10, 12, 15, 20, 22, 25, 30) # 3 players
cp1 <- mcstCorePoints(v = vvals1); cp1
cp1$vertices
plot(cp1)

# Example 2: Empty core
vvals2 <- c(10, 10, 10, 15, 15, 15, 30)
cp2 <- mcstCorePoints(v = vvals2); cp2

# Example 3: Using a game object
C <- matrix(c(0, 12, 15, 12, 0, 4, 15, 4, 0), byrow = TRUE, ncol = 3)
mcstCorePoints(game = mcstGamePrivate(C))
```

Description

This function computes Dutta-Kar's cost allocation (Dutta and Kar, 2004) for a minimum cost spanning tree problem (N_0, C) . The Dutta-Kar rule, denoted as $DK(N_0, C)$, is defined through Prim's algorithm with a pivotal switch in the allocation cost at each step to satisfy cost monotonicity.

Usage

```
mcstDuttaKar(
  C,
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node). For $n \leq 3$ with ties and method = "exact", it also displays all possible trees.
which.plot	character string indicating which plots to display (only for $n \leq 3$ with ties and method = "exact"). If "details", displays the detailed breakdown of all trees and allocations according to the agents' entry orders; if "main", the average allocation is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm, method (exact or Monte Carlo), and the total network cost.
method	character string specifying the calculation method when ties occur. If "exact" (default), computes all $n!$ permutations; if "montecarlo", estimates the average allocation through random sampling using nsim permutations (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

Following Prim's algorithm, let $x^0 = 0$. At each stage $p \in \{1, \dots, n\}$, the algorithm selects the minimum cost arc (i^p, j^p) that connects a new agent j^p to the already connected set of nodes. We update the accumulated cost as:

$$x^p = \max\{x^{p-1}, c_{i^p j^p}\}.$$

For each agent $i \in N$, there exists a stage $p(i)$ such that $i = j^{p(i)}$. Dutta-Kar's allocation for agent $i \in N$ is defined as:

$$DK_i(N_0, C) = \min\{x^{p(i)-1}, c_{i^{p(i)} j^{p(i)}}\}.$$

When the minimum cost spanning tree is not unique, the rule is extended by averaging the allocations over all possible permutations $\pi \in \Pi_N$ (method = "exact"):

$$DK(N_0, C) = \frac{1}{n!} \sum_{\pi \in \Pi_N} DK^\pi(N_0, C),$$

where $DK^\pi(N_0, C)$ is the allocation obtained by applying Prim's algorithm to (N_0, C) and solving indifference by selecting the first agent given by the permutation π .

Alternatively, `method = "montecarlo"` estimates the average allocation through random sampling using n_{sim} permutations. This is highly useful when the number of agents is large, as it significantly reduces computation time while providing a very accurate approximation:

$$DK(N_0, C) \approx \frac{1}{n_{\text{sim}}} \sum_{k=1}^{n_{\text{sim}}} DK^{\pi_k}(N_0, C),$$

where π_k represents each of the randomly sampled permutations. For networks with few agents, the exact method is preferred to avoid potential overestimations.

Value

A list containing:

- `dk`: the Dutta-Kar's allocation vector $DK(N_0, C)$ for the natural ordering.
- `e_dk`: the extended Dutta-Kar's allocation (average over permutations if ties exist; equal to `dk` otherwise).
- `arcs`: if the MCST is unique, a data frame of edges (i, j) in the tree and the stage at which they were added; if ties exist, a list of such data frames, one per distinct optimal tree.
- `total`: the total cost of the MCST, $m(N_0, C)$.
- `percentage`: the share of the total cost allocated to each agent, based on `e_dk`.
- `ranking`: a ranking of agents by cost in `e_dk` (from highest to lowest; ties marked with *).
- `perms`: if ties exist, a list with `$table` (allocations for at most the first 6 permutations computed, regardless of n), `$summary` (distinct allocations with their multiplicity, sorted in decreasing order) and `$nperms` (total number of permutations computed); NULL if the MCST is unique.
- `is_unique`: logical; if TRUE, the MCST is unique.
- `method`: the method applied for computing the extended rule ("exact" or "montecarlo").

Supported formats for C

Note: In all cases, `Inf` is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where `C[i, j]` represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a `.csv`, `.xlsx`, or `.xls` file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Dutta B, Kar A (2004) Cost monotonicity, consistency and minimum cost spanning tree games. *Games Econom Behav* 48(2):223–248

See Also

- [mcstBird](#) for other rule based on Prim's algorithm.
- [mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstDuttaKar(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstDuttaKar(C_inf)

# Matrix input with ties
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstDuttaKar(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstFolk

Folk Rule for MCSTP

Description

This function computes folk cost allocation (Feltkamp et al., 1994) for a minimum cost spanning tree problem (N_0, C) . The folk rule, denoted as $F(N_0, C)$, also known as the ERO (Equal Remaining Obligation) rule, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
mcstFolk(C, draw = FALSE, titles = TRUE)
```

Arguments

- | | |
|---------------------|--|
| <code>C</code> | cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below). |
| <code>draw</code> | logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node). |
| <code>titles</code> | logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost. |

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The folk rule is based on the obligation function:

$$o_i(S) = \frac{1}{|S|}.$$

At each stage p , let c_{ipjp} be the cost of the arc added to the network. The folk allocation for each agent $i \in N$ is given by:

$$F_i(N_0, C) = \sum_{p=1}^n c_{ipjp} \left(o_i(S_i^{p-1}) - o_i(S_i^p) \right).$$

Value

A list containing:

- folk: the folk allocation vector $F(N_0, C)$.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- is_unique: logical; if TRUE, the MCST is unique.

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The folk allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Feltkamp V, Tijs S, Muto S (1994) On the irreducible core and the equal remaining obligations rule of minimum cost spanning extension problems. Technical Report 106, CentER DP 1994, Tilburg University, The Netherlands

See Also

[mcstOWShapley](#), [mcstPWShapley](#) for other rules based on Kruskal's algorithm.

[mcstBoruvka](#) for an equivalent rule based on Boruvka's algorithm.

[mcstCone](#) with rule = "folk" for the equivalent implementation based on cone-wise decomposition.

[mcstGameIrred](#), [mcstGameOpt](#) for cooperative games whose Shapley value coincides with the folk solution.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstFolk(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstFolk(C_inf)

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstFolk(C_mat, draw = TRUE, titles = FALSE)
```

mcstGameCC

Cost Allocations based on the Cycle-complete Game for MCSTP

Description

This function computes the cost allocation for the cycle-complete game (Trudeau, 2012) associated with a minimum cost spanning tree problem (N_0, C') , using a specified solution concept. The cycle-complete game, denoted as (N, v_C^c) (or simply (N, v^c)), is defined as the private game associated with the cycle-complete network (N_0, C^*) .

Usage

```
mcstGameCC(
  C,
  sol = c("shapley", "nucleolus"),
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
sol	character; the solution concept to apply. Options are "shapley" (default), or "nucleolus" (only for method = "exact").
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.
which.plot	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.
method	character string specifying the calculation method. If "exact" (default), computes the exact allocation (Shapley value or nucleolus) and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using nsim permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

The cycle-complete game (N, v_C^c) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^c(S) = m(S_0, C^*),$$

where $m(S_0, C^*)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, evaluated over the cycle-complete costs C^* . The cycle-complete cost is defined as the minimum between the direct cost c_{ij} and the smallest value of the maximum arc cost over all cycles containing i and j , i.e., $c_{ij}^* = \min(c_{ij}, \min_{f \in \mathcal{C}(i,j)} \max_{e \in f} c_e)$, where $\mathcal{C}(i, j)$ denotes the set of all cycles that include both i and j .

For computational efficiency, C^* is obtained via the irreducible matrices $\bar{C}$ of the subgraphs as:

$$c_{ij}^* = \max_{k \in N \setminus \{i,j\}} \bar{c}_{ij}^{N \setminus \{k\}} \text{ for } i, j \in N,$$

and

$$c_{0i}^* = \max_{k \in N \setminus \{i\}} \bar{c}_{0i}^{N \setminus \{k\}} \text{ for } i \in N,$$

where $\bar{C}^{N \setminus \{k\}}$ is the irreducible cost matrix of the network excluding agent k . If no agents can be removed, the irreducible cost of the full network is used.

Under method = "exact", the function builds the full characteristic function (all $2^n - 1$ values). Depending on the chosen solution concept, it computes the allocation via [shapleyValue](#) (called *cycle-complete solution*, which relies on the coalition-based formula) or [nucleolus](#).

For larger networks, method = "montecarlo" provides an approximation of the Shapley value through random sampling using n_{sim} permutations. This approach implements lazy evaluation based on the permutation formula of the Shapley value: instead of computing the MST cost for all $2^n - 1$ possible subsets, it only evaluates the specific subcoalitions generated by the random permutations.

This is highly useful when the number of agents is large, as it significantly reduces memory usage and computation time while providing a very accurate approximation of the *cycle-complete solution*. For networks with few players, the exact method is preferred to avoid potential overestimations.

For more details on these formulas, see [mcstRules](#).

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$, ordered by cardinality. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **C_cc**: the cycle-complete cost matrix C^* .
- **v_c**: a numeric vector containing the characteristic function values $v^c(S)$ for all coalitions. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **contributions**: a data frame with the marginal contributions for the sampled permutations. If method = "exact", returns a string indicating that this table is not used.
- **allocation**: a numeric vector containing the computed cost allocation for each agent.
- **total**: the total cost of the MCST, $v^c(N)$.
- **nperms**: the number of permutations n_{sim} used in the calculation. If method = "exact", returns a string indicating that it was not computed.
- **percentage**: the share of the total cost allocated to each agent.
- **ranking**: ranking of agents by cost (from highest to lowest; ties marked with *).
- **method**: the method used for the calculation ("exact" or "montecarlo").
- **sol**: the solution concept used ("shapley" or "nucleolus").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The cycle-complete game is concave, which implies that its Shapley value (the *cycle-complete solution*) belongs to its core. Moreover, $\text{core}(N, v^c) \subseteq \text{core}(N, v^p)$.

References

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Trudeau C (2012) A new stable and more responsible cost sharing solution for mcst problems. *Games Econom Behav* 75(1):402–412

See Also

[mcstGamePrivate](#) for the pessimistic game based on the original costs.

[mcstGameIrred](#) for the pessimistic game based on the irreducible cost matrix $\bar{C}$.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstGameCC(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstGameCC(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstGameCC(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

Description

This function computes the cost allocation for the irreducible game (Bird, 1976) associated with a minimum cost spanning tree problem (N_0, C) , using a specified solution concept. The irreducible game, denoted as (N, v_C^i) (or simply (N, v^i)), is defined as the private game associated with the irreducible cost matrix C .

Usage

```
mcstGameIrred(
  C,
  sol = c("shapley", "nucleolus"),
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
sol	character; the solution concept to apply. Options are "shapley" (default), or "nucleolus" (only for method = "exact").
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.
which.plot	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.
method	character string specifying the calculation method. If "exact" (default), computes the exact allocation (Shapley value or nucleolus) and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using nsim permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

The irreducible game (N, v_C^i) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^i(S) = m(S_0, \bar{C}),$$

where $m(S_0, \bar{C})$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, evaluated over the irreducible costs $\bar{C}$. The irreducible cost between any two nodes corresponds to the maximum cost arc on the unique path connecting them in the optimal tree.

Under method = "exact", the function builds the full characteristic function (all $2^n - 1$ values). Depending on the chosen solution concept, it computes the allocation via [shapleyValue](#) (which relies on the coalition-based formula) or [nucleolus](#).

For larger networks, method = "montecarlo" provides an approximation of the Shapley value through random sampling using n_{sim} permutations. This approach implements lazy evaluation based on the permutation formula of the Shapley value: instead of computing the MST cost for all $2^n - 1$ possible subsets, it only evaluates the specific subcoalitions generated by the random permutations.

This is highly useful when the number of agents is large, as it significantly reduces memory usage and computation time while providing a very accurate approximation of the exact allocation. For networks with few players, the exact method is preferred to avoid potential overestimations.

For more details on these formulas, see [mcstRules](#).

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$, ordered by cardinality. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **C_irred**: the irreducible cost matrix $\bar{C}$ associated with the MCST of the problem.
- **v_i**: a numeric vector containing the characteristic function values $v^i(S)$ for all coalitions. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **contributions**: a data frame with the marginal contributions for the sampled permutations. If method = "exact", returns a string indicating that this table is not used.
- **allocation**: a numeric vector containing the computed cost allocation for each agent.
- **total**: the total cost of the MCST, $v^i(N)$.
- **nperms**: the number of permutations n_{sim} used in the calculation. If method = "exact", returns a string indicating that it was not computed.
- **percentage**: the share of the total cost allocated to each agent.
- **ranking**: ranking of agents by cost (from highest to lowest; ties marked with *).
- **method**: the method used for the calculation ("exact" or "montecarlo").
- **sol**: the solution concept used ("shapley" or "nucleolus").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The irreducible game is a strong reference for stability. Its core (the irreducible core) is always non-empty and constitutes a subset of the core of v^p . Classical rules such as Bird and folk are known to belong to the core of v^i .

It has been proven that the Shapley value of the irreducible game coincides with both the [mcstFolk](#) of the original problem and [mcstBird](#) applied to the irreducible costs (Bergantiños and Vidal-Puga, 2007a).

Unlike the private game, calculating solution concepts for the irreducible game is computationally efficient. Computing both the Shapley value and the nucleolus of v^i can be done in $O(|N|^2)$ time.

References

Bergantiños G, Vidal-Puga J (2007a) A fair rule in minimum cost spanning tree problems. *J Econom Theory* 137(1):326–352

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335–350

See Also

[mcstBird](#) of $(N_0, \bar{C})$, [mcstFolk](#) of (N_0, C) for classical rules belonging to the core of v^i that coincide with $Sh(N, v^i)$.

[mcstGamePrivate](#), [mcstGamePublic](#) for the pessimistic and optimistic games based on the original costs.

[mcstGameCC](#) for the pessimistic game based on the cycle-complete cost matrix C^* .

[mcstGameOpt](#) for an equivalent game under the Shapley value.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstGameIrred(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstGameIrred(C_inf, sol = "shapley")
```

```
# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstGameIrred(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstGameOpt

Cost Allocations based on the Optimistic Game for MCSTP

Description

This function computes the cost allocation for the optimistic game (Bergantiños and Vidal-Puga, 2007b) associated with a minimum cost spanning tree problem (N_0, C) , using a specified solution concept. The optimistic game, denoted as (N, v_C^o) (or simply (N, v^o)), evaluates the cost of each coalition $S \subseteq N$ assuming that agents in $N \setminus S$ are already connected. Thus, agents in S can connect to the source through agents in $N \setminus S$ for free.

Usage

```
mcstGameOpt(
  C,
  sol = c("shapley", "nucleolus"),
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

<code>C</code>	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
<code>sol</code>	character; the solution concept to apply. Options are "shapley" (default), or "nucleolus" (only for method = "exact").
<code>draw</code>	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.
<code>which.plot</code>	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is <code>c("main", "details")</code> .
<code>titles</code>	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.

method	character string specifying the calculation method. If "exact" (default), computes the exact allocation (Shapley value or nucleolus) and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using <code>nsim</code> permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

Let $T := N \setminus S$. The optimistic game (N, v_C^o) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^o(S) = m(S_0, C^T),$$

where $m(S_0, C^T)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source, using the modified cost matrix C^T . In this matrix, the cost between agents in S remains the same, i.e., $c_{ij}^T = c_{ij}$ for all $i, j \in S$, but the cost to connect any agent $i \in S$ to the source is updated as:

$$c_{0i}^T = \min_{j \in T \cup \{0\}} c_{ji}.$$

Under method = "exact", the function builds the full characteristic function (all $2^n - 1$ values). Depending on the chosen solution concept, it computes the allocation via [shapleyValue](#) (which relies on the coalition-based formula) or [nucleolus](#).

For larger networks, method = "montecarlo" provides an approximation of the Shapley value through random sampling using n_{sim} permutations. This approach implements lazy evaluation based on the permutation formula of the Shapley value: instead of computing the MST cost for all $2^n - 1$ possible subsets, it only evaluates the specific subcoalitions generated by the random permutations.

This is highly useful when the number of agents is large, as it significantly reduces memory usage and computation time while providing a very accurate approximation of the exact allocation. For networks with few players, the exact method is preferred to avoid potential overestimations.

For more details on these formulas, see [mcstRules](#).

Value

A list containing:

- `coalitions`: a character vector representing all possible coalitions $S \subseteq N$, ordered by cardinality. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- `C_opt`: a list of the optimistic cost matrices C^T for each coalition $S \subseteq N$. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- `v_o`: a numeric vector containing the characteristic function values $v^o(S)$ for all coalitions. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- `contributions`: a data frame with the marginal contributions for the sampled permutations. If method = "exact", returns a string indicating that this table is not used.
- `allocation`: a numeric vector containing the computed cost allocation for each agent.
- `total`: the total cost of the MCST, $v^o(N)$.

- `nperms`: the number of permutations `nsim` used in the calculation. If `method = "exact"`, returns a string indicating that it was not computed.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: ranking of agents by cost (from highest to lowest; ties marked with *).
- `method`: the method used for the calculation ("exact" or "montecarlo").
- `sol`: the solution concept used ("shapley" or "nucleolus").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where `C[i, j]` represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The optimistic game associated with any MCST problem coincides with the optimistic game associated with its irreducible form, i.e.,

$$v_C^o = v_{\bar{C}}^o.$$

If $(N_0, \bar{C})$ is irreducible, the private and optimistic games, v^p and v^o , are dual: $v^p(S) + v^o(N \setminus S) = m(N_0, C)$ for all $S \subset N$.

Furthermore, $Sh(N, v_C^o) = Sh(N, v_{\bar{C}}^o)$, so applying the Shapley value to v^o provides an alternative way of obtaining the [mcstFolk](#) (Bergantiños and Vidal-Puga, 2007b).

References

Bergantiños G, Vidal-Puga J (2007b) The optimistic TU game in minimum cost spanning tree problems. *Int J Game Theory* 36(2):223–239

Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[mcstGamePrivate](#) for a game based on a pessimistic approach.

[mcstGameIrred](#) for an equivalent game under the Shapley value.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstGameOpt(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstGameOpt(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstGameOpt(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstGamePrivate

Cost Allocations based on the Private Game for MCSTP

Description

This function computes the cost allocation for the private or pessimistic game (Bird, 1976) associated with a minimum cost spanning tree problem (N_0, C) , using a specified solution concept. The private game, denoted as (N, v_C^p) (or simply (N, v^p)), evaluates the cost of each coalition $S \subseteq N$ under the assumption that agents outside the coalition are not available to provide their nodes as intermediaries.

Usage

```
mcstGamePrivate(
  C,
  sol = c("shapley", "nucleolus"),
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
sol	character; the solution concept to apply. Options are "shapley" (default), or "nucleolus" (only for method = "exact").
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.

which.plot	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.
method	character string specifying the calculation method. If "exact" (default), computes the exact allocation (Shapley value or nucleolus) and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using n_{sim} permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

The private game (N, v_C^p) is a cooperative game with transferable utility (TU game) defined for each coalition $S \subseteq N$ as:

$$v_C^p(S) = m(S_0, C),$$

where $m(S_0, C)$ is the cost of the minimum spanning tree of the subproblem induced by the agents in S and the source. This approach is termed "private" or "pessimistic" because the nodes in $N \setminus S$ belong to these agents, and their participation is needed in order to use their nodes.

When the Shapley value (sol = "shapley") is applied to this game, the resulting allocation is known as Kar's rule (Kar, 2002). For a function that particularizes this specific case, see [mcstKar](#).

Under method = "exact", the function builds the full characteristic function (all $2^n - 1$ values). Depending on the chosen solution concept, it computes the allocation via [shapleyValue](#) (which relies on the coalition-based formula) or [nucleolus](#).

For larger networks, method = "montecarlo" provides an approximation of the Shapley value through random sampling using n_{sim} permutations. This approach implements lazy evaluation based on the permutation formula of the Shapley value: instead of computing the MST cost for all $2^n - 1$ possible subsets, it only evaluates the specific subcoalitions generated by the random permutations.

This is highly useful when the number of agents is large, as it significantly reduces memory usage and computation time while providing a very accurate approximation of Kar's rule. For networks with few players, the exact method is preferred to avoid potential overestimations.

For more details on these formulas, see [mcstRules](#).

Value

A list containing:

- **coalitions**: a character vector representing all possible coalitions $S \subseteq N$, ordered by cardinality. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **v_p**: a numeric vector containing the characteristic function values $v^p(S)$ for all coalitions. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- **contributions**: a data frame with the marginal contributions for the sampled permutations. If method = "exact", returns a string indicating that this table is not used.

- allocation: a numeric vector containing the computed cost allocation for each agent.
- total: the total cost of the MCST, $v^p(N)$.
- nperms: the number of permutations nsim used in the calculation. If method = "exact", returns a string indicating that it was not computed.
- percentage: the share of the total cost allocated to each agent.
- ranking: ranking of agents by cost (from highest to lowest; ties marked with *).
- method: the method used for the calculation ("exact" or "montecarlo").
- sol: the solution concept used ("shapley" or "nucleolus").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The private game is a strong reference for stability. Classical rules such as Bird, Dutta-Kar, or the folk rule are known to belong to the core of v^p for any MCST problem. As a result, these allocations are coalitionally stable under the pessimistic cost perspective.

Although the Shapley value and the nucleolus are well-established solution concepts, it has been proven that computing both for this game is NP-hard.

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Bird C (1976) On cost allocation for a spanning tree: a game theoretic approach. *Networks* 6(4):335–350
- Kar A (2002) Axiomatization of the Shapley value on minimum cost spanning tree games. *Games Econom Behav* 38(2):265–277

See Also

[mcstBird](#), [mcstDuttaKar](#), [mcstFolk](#) for classical rules belonging to the core of v^p .
[mcstKar](#) for the specific cost allocation rule based on the Shapley value of this game.
[mcstGameIrred](#), [mcstGameCC](#) for the pessimistic games based on the irreducible cost matrix $\bar{C}$ and the cycle-complete cost matrix C^* , respectively.

`mcstGameOpt`, `mcstGamePublic` for optimistic alternatives to the private approach.
`mcstRules` for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstGamePrivate(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstGamePrivate(C_inf, sol = "nucleolus")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstGamePrivate(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstGamePublic

Cost Allocations based on the Public Game for MCSTP

Description

This function computes the cost allocation for the public game (Bogomolnaia and Moulin, 2010) associated with a minimum cost spanning tree problem (N_0, C) , using a specified solution concept. The public game, denoted as (N, v_C^u) (or simply (N, v^u)), evaluates the cost of each coalition $S \subseteq N$ assuming that there are no property rights on nodes. Therefore, agents in S can use the nodes of agents in $N \setminus S$ to connect to the source, paying the corresponding arc costs.

Usage

```
mcstGamePublic(
  C,
  sol = c("shapley", "nucleolus"),
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

<code>C</code>	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
<code>sol</code>	character; the solution concept to apply. Options are "shapley" (default), or "nucleolus" (only for method = "exact").

draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.
which.plot	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.
method	character string specifying the calculation method. If "exact" (default), computes the exact allocation (Shapley value or nucleolus) and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using <code>nsim</code> permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Details

The public game (N, v_C^u) is a cooperative game with transferable utility (TU game) where the cost for each coalition $S \subseteq N$ is defined as the minimum private game cost among all its supersets $T \supseteq S$, i.e.:

$$v_C^u(S) = \min_{S \subseteq T} v_C^p(T),$$

where $v_C^p(\cdot)$ is the characteristic function of the private game. This means computing $v^u(S)$ requires calculating the private game costs for all possible supersets of S .

Under method = "exact", the function builds the full characteristic function (all $2^n - 1$ values). Depending on the chosen solution concept, it computes the allocation via [shapleyValue](#) (which relies on the coalition-based formula) or [nucleolus](#).

For larger networks, method = "montecarlo" provides an approximation of the Shapley value through random sampling using n_{sim} permutations. This approach implements lazy evaluation based on the permutation formula of the Shapley value: instead of computing the MST cost for all $2^n - 1$ possible subsets, it only evaluates the specific subcoalitions generated by the random permutations.

This is highly useful when the number of agents is large, as it significantly reduces memory usage and computation time while providing a very accurate approximation of the exact allocation. For networks with few players, the exact method is preferred to avoid potential overestimations.

For more details on these formulas, see [mcstRules](#).

Value

A list containing:

- `coalitions`: a character vector representing all possible coalitions $S \subseteq N$, ordered by cardinality. If method = "montecarlo", returns a string indicating that lazy evaluation was used.
- `v_u`: a numeric vector containing the characteristic function values $v^u(S)$ for all coalitions. If method = "montecarlo", returns a string indicating that lazy evaluation was used.

- `v_p`: a numeric vector containing the characteristic function values $v^p(S)$ for all coalitions. If `method = "montecarlo"`, returns a string indicating that lazy evaluation was used.
- `best_T`: a data frame mapping each coalition S to the optimal superset T that minimizes its cost. If `method = "montecarlo"`, returns a string indicating that lazy evaluation was used.
- `contributions`: a data frame with the marginal contributions for the sampled permutations. If `method = "exact"`, returns a string indicating that this table is not used.
- `allocation`: a numeric vector containing the computed cost allocation for each agent.
- `total`: the total cost of the MCST, $v^u(N)$.
- `nperms`: the number of permutations `nsim` used in the calculation. If `method = "exact"`, returns a string indicating that it was not computed.
- `percentage`: the share of the total cost allocated to each agent.
- `ranking`: ranking of agents by cost (from highest to lowest; ties marked with *).
- `method`: the method used for the calculation ("exact" or "montecarlo").
- `sol`: the solution concept used ("shapley" or "nucleolus").

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where `C[i, j]` represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The core of v^u coincides with the core of v^p restricted to nonnegative cost shares. Consequently, classical rules such as Bird, Dutta-Kar, or the folk rule belong to the core of the public game for any MCST problem.

However, unlike in the private game, the Shapley value of the public game does not necessarily belong to the core of v^u (Trudeau and Vidal-Puga, 2019).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Bogomolnaia A, Moulin H (2010) Sharing a minimal cost spanning tree: beyond the folk solution. *Games and Econom Behav* 69(2):238–248
- Trudeau C, Vidal-Puga J (2019) The Shapley value in minimum cost spanning tree problems. In: Algaba E, Fragnelli V, Sánchez-Soriano J (eds) *Chapters in Game Theory: The Shapley value*, chapter 24. CRC Press, Taylor & Francis Group

See Also

[mcstBird](#), [mcstDuttaKar](#), [mcstFolk](#) for classical rules belonging to the core of v^u .
[mcstGamePrivate](#) for the pessimistic alternative to the public game.
[mcstGameIrred](#) for a game that can be defined from either the private or the public approach.
[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstGamePublic(c(12, 15, 20, 4, 6, 8), sol = "shapley", draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstGamePublic(C_inf, sol = "shapley")

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstGamePublic(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstKar

Kar's Rule for MCSTP

Description

This function computes Kar's rule for a minimum cost spanning tree problem (N_0, C) . It is defined as the Shapley value of the associated private game (Kar, 2002).

Usage

```
mcstKar(
  C,
  draw = FALSE,
  which.plot = c("main", "details"),
  titles = TRUE,
  method = c("exact", "montecarlo"),
  nsim = 10000
)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see the "Supported formats for C" section in mcstGamePrivate).
draw	logical; if TRUE, plots the network highlighting an optimal tree in red (constructed using Prim's algorithm), indicating the cost allocated to each agent in brackets below the node. For $n \leq 3$ and method = "exact", it also displays the optimal network for each subcoalition $S \subset N$.

which.plot	character string indicating which plots to display. If "details" (only for $n \leq 3$ and method = "exact"), displays the optimal network for each subcoalition $S \subset N$; if "main", the allocation for N is plotted. Default is c("main", "details").
titles	logical; if TRUE (default), adds a main title specifying the cooperative game and a subtitle with the solution concept, method (exact or Monte Carlo) and the total network cost.
method	character string specifying the calculation method. If "exact" (default), computes the exact Shapley value allocation and the full characteristic function; if "montecarlo", estimates an approximation of the Shapley value through random sampling using nsim permutations. It uses lazy evaluation to avoid computing all possible coalitions (recommended for large networks).
nsim	integer; the number of permutations to sample when using method = "montecarlo". Default is 10,000.

Value

A list containing the computed allocation and calculation details. For a comprehensive breakdown of the elements returned in this list, see the "Value" section in [mcstGamePrivate](#).

Note

For an in-depth explanation of the private game's properties, its characteristic function formulation, and discussions on computational complexity, please refer to the documentation for [mcstGamePrivate](#).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Kar A (2002) Axiomatization of the Shapley value on minimum cost spanning tree games. *Games Econom Behav* 38(2):265–277

See Also

[mcstGamePrivate](#) for the general pessimistic game.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input
mcstKar(c(12, 15, 20, 4, 6, 8), draw = TRUE)

# Input with infinite costs (disconnected nodes)
C_inf <- c(5, 9, Inf, 15, 6, Inf, 7, Inf, Inf, Inf,
          Inf, 8, 7, Inf, Inf, 5, Inf, Inf, 8, 9, 11)
mcstKar(C_inf)

# Matrix input
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstKar(C_mat, draw = TRUE, which.plot = "main", titles = FALSE)
```

mcstOWShapley

*Optimistic Weighted Shapley's Rule for MCSTP***Description**

This function computes the optimistic weighted Shapley cost allocation (Bergantiños and Lorenzo-Freire, 2008a,b) for a minimum cost spanning tree problem (N_0, C) . The optimistic weighted Shapley rule, denoted as $f^{\ell^{ow}}(N_0, C)$, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
mcstOWShapley(C, weights, draw = FALSE, titles = TRUE)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The optimistic weighted Shapley rule is based on the obligation function:

$$o_i(S) = \frac{w_i}{\sum_{j \in S} w_j},$$

where $w_i > 0$ is the weight associated with agent i .

At each stage p , let $c_{i^p j^p}$ be the cost of the arc added to the network. The optimistic weighted Shapley allocation for each agent $i \in N$ is given by:

$$f_i^{\ell^{ow}}(N_0, C) = \sum_{p=1}^n c_{i^p j^p} \left(o_i(S_i^{p-1}) - o_i(S_i^p) \right).$$

Value

A list containing:

- **ows**: the optimistic weighted Shapley allocation vector $f^{\ell^{ow}}(N_0, C)$.
- **weights**: the weight vector used for the allocation.
- **arcs**: a data frame of edges (i, j) in the final tree and the stage at which they were added.

- **total**: the total cost of the MCST, $m(N_0, C)$.
- **percentage**: the share of the total cost allocated to each agent.
- **ranking**: a ranking of agents by cost (from highest to lowest; ties marked with *).
- **is_unique**: logical; if TRUE, the MCST is unique.

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n + 1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The optimistic weighted Shapley allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

Furthermore, when all agents share the same weight, the optimistic weighted Shapley allocation is mathematically equivalent to the [mcstFolk](#).

References

- Bergantiños G, Lorenzo-Freire S (2008a) A characterization of optimistic weighted Shapley rules in minimum cost spanning tree problems. *Econ Theor* 35(3):523–538
- Bergantiños G, Lorenzo-Freire S (2008b) "Optimistic" weighted Shapley rules in minimum cost spanning tree problems. *European J Operatl Res* 185(1):289–298
- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100

See Also

[mcstFolk](#), [mcstPWShapley](#) for other rules based on Kruskal's algorithm.

[mcstCone](#) with rule = "owshapley" for the equivalent implementation based on cone-wise decomposition.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input with custom weights
mcstOWShapley(c(12, 15, 20, 4, 6, 8), weights = c(2, 1, 1))

# Matrix input with equal weights (equivalent to folk rule)
C_mat <- matrix(c(0, 12, 15, 12,
                  12, 0, 4, 6,
                  15, 4, 0, 8,
                  12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstOWShapley(C_mat, weights = rep(1,3), draw = TRUE, titles = FALSE)
```

mcstPWShapley

Pessimistic Weighted Shapley's Rule for MCSTP

Description

This function computes the pessimistic weighted Shapley cost allocation (Lorenzo and Lorenzo-Freire, 2009) for a minimum cost spanning tree problem (N_0, C) . The pessimistic weighted Shapley rule, denoted as $f^{e^{pw}}(N_0, C)$, is defined through Kruskal's algorithm using a function with specific properties, called the obligation function $o_i(S)$.

Usage

```
mcstPWShapley(C, weights, draw = FALSE, titles = TRUE)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" below).
weights	a numeric vector of strictly positive weights for each agent in N . The length of the vector must match the number of agents.
draw	logical; if TRUE, plots the network highlighting an optimal tree in red, indicating the stage at which each arc is added (in brackets) and the cost allocated to each agent (in parentheses below the node).
titles	logical; if TRUE (default), adds a main title specifying the allocation rule and a subtitle with the algorithm used and the total network cost.

Details

Following Kruskal's algorithm, let g^p be the network at stage $p \in \{1, \dots, n\}$, and $S_i^p := S(P(g^p), i)$ the connected component containing agent i . The pessimistic weighted Shapley rule is based on the obligation function:

$$o_i(S) = \sum_{\pi \in \Pi(S \setminus \{i\})} \prod_{j=1}^{|S|-1} \frac{w_{\pi^{-1}(j)}}{\sum_{k=1}^j w_{\pi^{-1}(k)} + w_i},$$

where $\Pi(\cdot)$ denotes the set of all permutations of a given set, and $w_i > 0$ is the weight associated with agent i .

At each stage p , let c_{ij^p} be the cost of the arc added to the network. The pessimistic weighted Shapley allocation for each agent $i \in N$ is given by:

$$f_i^{g^{pw}}(N_0, C) = \sum_{p=1}^n c_{ij^p} (o_i(S_i^{p-1}) - o_i(S_i^p)).$$

Value

A list containing:

- pws: the pessimistic weighted Shapley allocation vector $f^{g^{pw}}(N_0, C)$.
- weights: the weight vector used for the allocation.
- arcs: a data frame of edges (i, j) in the final tree and the stage at which they were added.
- total: the total cost of the MCST, $m(N_0, C)$.
- percentage: the share of the total cost allocated to each agent.
- ranking: a ranking of agents by cost (from highest to lowest; ties marked with *).
- is_unique: logical; if TRUE, the MCST is unique.

Supported formats for C

Note: In all cases, Inf is accepted for disconnected node pairs. The first row/column is assumed to be the source node (0).

The function accepts the following formats for the cost input:

Numeric vector The lower triangle of a symmetric cost matrix, excluding the diagonal. Length must be $n(n+1)/2$, where n is the number of agents (excluding the source).

Square matrix / Data frame A standard adjacency matrix where $C[i, j]$ represents the cost between node i and node j .

Edge list (matrix or data frame) An alternative to the full matrix. It must contain columns named "from" and "to", plus a third column for the weights/costs.

'igraph' object A weighted undirected graph. See the [igraph](#) package for details on creating these objects.

File path A string pointing to a .csv, .xlsx, or .xls file. The file can contain either a square matrix (without headers) or an edge list (with "from" and "to" headers).

Note

The pessimistic weighted Shapley allocation is uniquely determined even when the minimum cost spanning tree is not unique. Since the rule depends on the evolution of the connected components rather than the specific arcs selected, any tie-breaking selection in Kruskal's algorithm yields the same cost distribution.

Furthermore, when all agents share the same weight, the pessimistic weighted Shapley allocation is mathematically equivalent to the [mcstFolk](#).

References

- Bergantiños G, Vidal-Puga J (2021) A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100
- Lorenzo L, Lorenzo-Freire S (2009) A characterization of Kruskal sharing rules for minimum cost spanning tree problems. *Int J Game Theory* 38(1):107-126

See Also

[mcstFolk](#), [mcstOWShapley](#) for other rules based on Kruskal's algorithm.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Simple vector input with custom weights
mcstPWShapley(c(12, 15, 20, 4, 6, 8), weights = c(2, 1, 1))

# Matrix input with equal weights (equivalent to folk rule)
C_mat <- matrix(c(0, 12, 15, 12,
                 12, 0, 4, 6,
                 15, 4, 0, 8,
                 12, 6, 8, 0), byrow = TRUE, ncol = 4)
mcstPWShapley(C_mat, weights = rep(1,3), draw = TRUE, titles = FALSE)
```

mcstRules

Overview of Rules and Analysis Tools for Minimum Cost Spanning Tree Problems

Description

This page provides an overview of the cost allocation rules implemented in this package for minimum cost spanning tree problems (MCSTP). The rules are organized into two main approaches: those defined directly through algorithms and decomposition techniques, and those defined through the analysis of associated cooperative games. It also includes a suite of analysis tools.

Rules defined through the problem

These rules are obtained directly from the network's structure and classical MST algorithms.

Based on Prim's algorithm:

These rules follow the growth of a single tree starting from the source.

- [mcstBird](#): agents sequentially connect to the growing tree, paying the cost of the arc through which they connect to the tree.
- [mcstDuttaKar](#): a modification of Bird's rule that introduces a pivotal switch in the allocation cost at each step to ensure cost monotonicity.

Based on Kruskal's algorithm:

Based on obligation functions, these rules follow Kruskal's logic of merging connected components using the cheapest available arcs.

- [mcstFolk](#): the agents divide the cost of connecting their components equally (ERO rule).
- [mcstOWShapley](#): optimistic weighted Shapley rule. A generalization of the folk rule that distributes costs proportionally based on asymmetric positive weights.
- [mcstPWShapley](#): pessimistic weighted Shapley rule. An alternative weighted extension based on a permutation-driven obligation function.

Based on Boruvka's algorithm:

- **mcstBoruvka**: the agents pay the maximum possible proportion of the cheapest arc selected by their component. The numerical outcome is mathematically equivalent to the **mcstFolk**.

Based on a cone-wise decomposition:

- **mcstCone**: a technique that decomposes any general MCSTP into a nonnegative combination of elementary problems (where costs are 0 or 1). It allows extending rules like "folk", "owshapley", or the Bogomolnaia and Moulin family ("bogomolnaia") to general networks.

Rules defined through cooperative games

These rules are defined by first associating a cooperative game (in characteristic function form v) to the MCSTP and then applying a solution concept (like the Shapley value or the Nucleolus).

Cooperative games:

- **mcstGamePrivate**: defines the cost of a coalition as the MST cost of the subproblem formed by its members and the source (pessimistic approach).
- **mcstKar**: defined as the Shapley value of the associated private game.
- **mcstGameIrred**: based on the irreducible matrix $\bar{C}$, where costs reflect the cheapest way to connect each pair of nodes through the network.
- **mcstGameOpt**: allows a coalition to connect through nodes outside the coalition at no cost.
- **mcstGamePublic**: assigns to each coalition the minimum cost achievable when connections through other agents are publicly available.
- **mcstGameCC**: based on the cycle-complete matrix C^* , which induces a concave cooperative game.

Solution concepts:

- Shapley value:
 - Based on permutations: the average of marginal contributions over all $n!$ possible orderings (Π_N):

$$Sh_i(N, v) = \frac{1}{n!} \sum_{\pi \in \Pi_N} [v(Pre(i, \pi) \cup \{i\}) - v(Pre(i, \pi))],$$

where $Pre(i, \pi)$ is the set of players that precede agent i in the ordering π .

- Based on coalitions: a weighted sum of marginal contributions over all possible subcoalitions S that do not contain i :

$$Sh_i(N, v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

The Shapley value is the unique solution that satisfies the properties of efficiency, symmetry, additivity, and the null player property.

- Nucleolus: the unique cost allocation x that lexicographically minimizes the maximum excesses (complaints) of all coalitions. In a cost game, the excess e of a coalition S measures its dissatisfaction:

$$e(S, x) = \sum_{j \in S} x_j - v(S).$$

The nucleolus ensures that the most dissatisfied coalition is as happy as possible. It is always efficient and lies within the Core if the Core is non-empty.

Analysis tools

The package includes a comprehensive suite of tools to analyze the stability, sensitivity, and geometric properties of the cost allocations and the network:

- `mcstCoreCheck`: evaluates whether a given cost allocation belongs to the core of a cooperative game, identifying any blocking coalitions if unstable.
- `mcstCorePoints`: determines if the core of a game is empty via Linear Programming, computes a feasible stable allocation, and finds the exact geometric vertices of the core polytope.
- `mcstCorePlot`: generates geometric visualizations of the core region and the imputation set for 2, 3, or 4 players, allowing for a graphical stability check.
- `mcstStabilityRange`: calculates the stability range (lower and upper bounds) for the cost of individual arcs without altering the optimal tree topology.
- `mcstSensitivity`: performs sensitivity analysis by varying the costs of specific arcs (individually, jointly, or independently) and observing the dynamic impact on the agents' allocations.
- `mcstCompare`: computes and compares the cost allocations of four standard algorithmic rules (Bird, Dutta-Kar, folk, and Kar) simultaneously.

Note

When the optimal tree is not unique, rules based on Prim's algorithm compute the symmetric average allocation over all possible permutations. In contrast, rules based on Kruskal's and Boruvka's algorithms, as well as the cone-wise decomposition, are unaffected by tie-breaking choices.

For game-theoretic rules, the functions in this package can compute either the Shapley value or the Nucleolus of the associated cooperative game.

References

Bergantiños, G., & Vidal-Puga, J. (2021). A review of cooperative rules and their associated algorithms for minimum-cost spanning tree problems. *SERIEs*, 12:73-100.

mcstSensitivity

Sensitivity Analysis for Cost Allocations

Description

This function performs a sensitivity analysis on cost allocation rules for a minimum cost spanning tree (MCST) problem by varying the costs of specific arcs and observing the impact on the agents' cost allocation.

Usage

```
mcstSensitivity(
  C,
  rule = c("bird", "duttakar", "folk", "kar"),
  arcs,
  delta = list(c(-5, 5), c(-3, 3)),
  step = 1,
  independent = FALSE
)

## S3 method for class 'mcstp_sens'
plot(x, titles = TRUE, ...)
```

Arguments

<code>C</code>	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" in mcstBird).
<code>rule</code>	character string specifying the allocation rule to evaluate. Must be one of "bird", "duttakar", "folk", or "kar".
<code>arcs</code>	a vector of length 2 specifying a single arc (i, j) , or a list of vectors specifying multiple arcs to analyze.
<code>delta</code>	a numeric vector specifying the lower and upper limits of cost variation $[\Delta LB, \Delta UB]$, or a list of two vectors if analyzing two independent arcs. Default is <code>list(c(-5, 5), c(-3, 3))</code> .
<code>step</code>	numeric value specifying the step size for the sequence of variations. Default is 1.
<code>independent</code>	logical; if TRUE, evaluates the cross-sensitivity of exactly two arcs independently, generating an evaluation grid. Default is FALSE.
<code>x</code>	the output from <code>mcstSensitivity</code> .
<code>titles</code>	logical; if TRUE (default), adds informative main titles and axis labels to the plots.
<code>...</code>	additional graphical parameters passed to the plot function.

Details

The sensitivity analysis can be conducted under three different scenarios depending on the structure of the arcs parameter and the independent flag:

1. Single arc variation: when a single arc is provided, its cost varies along the designated `delta` interval. The output plot displays evolution lines for each agent's cost allocation relative to the absolute cost of the arc.
2. Joint arc variation: when multiple arcs are provided and `independent = FALSE`, the same `delta` variation value is applied simultaneously to all selected arcs.
3. Independent variation of two arcs: when exactly two arcs are provided and `independent = TRUE`, a grid of cross-variations is constructed using `delta1` and `delta2`. The output plot renders a selection of heatmaps (one per agent) mapping the allocations across the variation space.

Value

A list containing:

- `delta` (or `delta1` and `delta2`): the variation step values applied.
- `arc_cost`: the resulting absolute cost of the arc (computed only in the single arc case).
- Columns named after each agent showing the dynamic cost allocations obtained under the selected rule.

See Also

[mcstBird](#), [mcstDuttaKar](#), [mcstFolk](#), [mcstKar](#) for the rule definitions.

[mcstStabilityRange](#) for calculating static boundaries.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Matrix input
C_mat <- matrix(c(0, 10, 15, 20,
                 10, 0, 25, 12,
                 15, 25, 0, 8,
                 20, 12, 8, 0), byrow = TRUE, ncol = 4)

# Example 1: Single arc sensitivity analysis
sens_single <- mcstSensitivity(C_mat, rule = "bird", arcs = c("0", "1"), delta = c(-5, 15))
plot(sens_single)

# 2. Joint sensitivity analysis for multiple arcs
arcs_list <- list(c("0", "1"), c("2", "3"))
sens_joint <- mcstSensitivity(C_mat, rule = "folk", arcs = arcs_list, delta = c(-3, 3))
plot(sens_joint)
```

mcstStabilityRange	<i>Stability Range for MCST Arcs</i>
--------------------	--------------------------------------

Description

This function calculates the stability range for the arcs of a minimum cost spanning tree (MCST). It determines how much the cost of an individual arc can increase or decrease without changing the topology of the optimal tree.

Usage

```
mcstStabilityRange(C)
```

Arguments

C	cost matrix between nodes. Accepts multiple formats (see "Supported formats for C" in mcstBird).
---	---

Details

The stability range of an arc (i, j) indicates the interval $[LB, UB]$ within which its original cost c_{ij} can vary while preserving the current minimum cost spanning tree as optimal.

At the finite boundary points of the stability interval, the minimum cost spanning tree becomes non-unique due to cost ties. In these scenarios, component or path-based rules (e.g., Folk, Kar) remain stable, whereas permutation-based rules such as Bird's rule or Dutta-Kar's rule average allocations over all alternative optimal trees, which can induce non-proportional jumps in the final cost allocation vector.

Value

A data frame containing:

- from: the starting node of the arc.
- to: the ending node of the arc.
- in_mst: logical; if TRUE, the arc belongs to the optimal tree.

- cost: the original cost of the arc c_{ij} .
- range: a formatted string showing the allowed cost variation $[\Delta LB, \Delta UB]$.
- LB: the absolute lower bound for the arc's cost.
- UB: the absolute upper bound for the arc's cost.

See Also

[mcstBird](#), [mcstDuttaKar](#), [mcstFolk](#), [mcstKar](#) for allocation rules.

[mcstRules](#) for an overview of the available rules and analysis tools in the package.

Examples

```
# Matrix input
C_mat <- matrix(c(0, 10, 15, 20,
                  10, 0, 25, 12,
                  15, 25, 0, 8,
                  20, 12, 8, 0), byrow = TRUE, ncol = 4)

# Calculate stability range
mcstStabilityRange(C_mat)
```

Index

* Algorithmic Rules

mcstBird, [2](#)
mcstBoruvka, [5](#)
mcstCone, [8](#)
mcstDuttaKar, [15](#)
mcstFolk, [18](#)
mcstOWShapley, [38](#)
mcstPWSchapley, [40](#)
mcstRules, [42](#)

* Analysis Tools

mcstRules, [42](#)

* Comparison

mcstCompare, [7](#)

* Cooperative Games

mcstCoreCheck, [10](#)
mcstCorePlot, [12](#)
mcstCorePoints, [14](#)
mcstGameCC, [20](#)
mcstGameIrred, [24](#)
mcstGameOpt, [27](#)
mcstGamePrivate, [30](#)
mcstGamePublic, [33](#)
mcstKar, [36](#)
mcstRules, [42](#)

* Core Stability

mcstCoreCheck, [10](#)
mcstCorePlot, [12](#)
mcstCorePoints, [14](#)

* Linear Programming

mcstCorePoints, [14](#)

* MCSTP

mcstBird, [2](#)
mcstBoruvka, [5](#)
mcstCompare, [7](#)
mcstCone, [8](#)
mcstDuttaKar, [15](#)
mcstFolk, [18](#)
mcstGameCC, [20](#)
mcstGameIrred, [24](#)
mcstGameOpt, [27](#)
mcstGamePrivate, [30](#)
mcstGamePublic, [33](#)
mcstKar, [36](#)

mcstOWShapley, [38](#)
mcstPWSchapley, [40](#)
mcstRules, [42](#)
mcstSensitivity, [44](#)
mcstStabilityRange, [46](#)

* Sensitivity Analysis

mcstSensitivity, [44](#)
mcstStabilityRange, [46](#)

* Visualization

mcstCorePlot, [12](#)

barplot, [7](#), [11](#)

igraph, [4](#), [6](#), [10](#), [17](#), [19](#), [23](#), [26](#), [29](#), [32](#), [35](#), [39](#),
[41](#)

mcstBird, [2](#), [7](#), [18](#), [26](#), [32](#), [36](#), [42](#), [45–47](#)

mcstBoruvka, [5](#), [20](#), [43](#)

mcstCompare, [7](#), [44](#)

mcstCone, [8](#), [20](#), [39](#), [43](#)

mcstCoreCheck, [10](#), [13](#), [15](#), [44](#)

mcstCorePlot, [12](#), [14](#), [15](#), [44](#)

mcstCorePoints, [14](#), [44](#)

mcstDuttaKar, [4](#), [7](#), [15](#), [32](#), [36](#), [42](#), [45](#), [47](#)

mcstFolk, [6](#), [7](#), [10](#), [18](#), [26](#), [29](#), [32](#), [36](#), [39](#),
[41–43](#), [45](#), [47](#)

mcstGameCC, [20](#), [26](#), [32](#), [43](#)

mcstGameIrred, [4](#), [12](#), [20](#), [23](#), [24](#), [29](#), [32](#), [36](#),
[43](#)

mcstGameOpt, [20](#), [26](#), [27](#), [33](#), [43](#)

mcstGamePrivate, [11–14](#), [23](#), [26](#), [29](#), [30](#), [36](#),
[37](#), [43](#)

mcstGamePublic, [26](#), [33](#), [33](#), [43](#)

mcstKar, [7](#), [31](#), [32](#), [36](#), [43](#), [45](#), [47](#)

mcstOWShapley, [10](#), [20](#), [38](#), [42](#)

mcstPWSchapley, [20](#), [39](#), [40](#), [42](#)

mcstRules, [4](#), [6](#), [7](#), [10](#), [12](#), [13](#), [15](#), [18](#), [20](#), [22](#),
[23](#), [25](#), [26](#), [28](#), [29](#), [31](#), [33](#), [34](#), [36](#), [37](#),
[39](#), [42](#), [42](#), [45](#), [47](#)

mcstSensitivity, [44](#), [44](#)

mcstStabilityRange, [44](#), [45](#), [46](#)

nucleolus, [22](#), [25](#), [28](#), [31](#), [34](#)

plot.mcstp_compare (mcstCompare), [7](#)

`plot.mcstp_core` (`mcstCoreCheck`), [10](#)
`plot.mcstp_core_point` (`mcstCorePoints`),
[14](#)
`plot.mcstp_sens` (`mcstSensitivity`), [44](#)
`shapleyValue`, [22](#), [25](#), [28](#), [31](#), [34](#)